TracerX-Pruning Dynamic Symbolic Execution with Weakest Precondition Interpolation

Arpita Dutta[1], Rasool Maghareh[2], and Joxan Jaffar[3]

School of Computing, National University of Singapore, Singapore[1,3],Huawei Canada Research Centre, Canada[2]

TracerX Framework

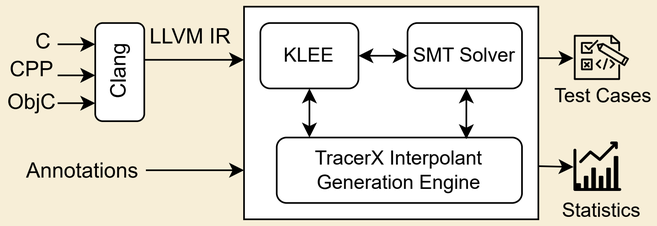


Figure 1. The overall architecture of TracerX

Highlights

- TracerX is Dynamic Symbolic Executor (DSE) which is built upon KLEE [1].
- The major advantage of DSE is its path-by-path exploration of the program execution space. However, this often leads to the path explosion problem.
- To address this issue, a method of abstraction learning has been used. The key step here is the computation of an interpolant to represent the learned abstraction [2].
- We use two different approaches of interpolant generation viz., 1. Deletion Interpolation [3] and 2. Weakest Precondition (WP) Interpolation [4]
- Deletion Interpolation (TracerX-Del) is our more stable and mature system and is briefly discussed in [3].
- Here, we present the Weakest Precondition (WP) Interpolation approach for TracerX, i.e., TracerX-WP.
- WP interpolation provides a more general interpolant which can have a higher chance of subsuming more subtrees in SET.
- Our primary conclusion is that we can efficiently reach completeness in the SET search.

From KLEE (No Interpolation) to TracerX (With Interpolation)

- Forward Symbolic Execution to find feasible paths (Similar to KLEE).
- Intermediate execution states preserved (Unlike KLEE).
- Half interpolants are generated by backward tracking and Full interpolants generated by merging half interpolants.
- Full interpolants used for subsumption at similar program points.
- TracerX uses information from already traversed (symbolic execution) subtree to prune other subtrees.

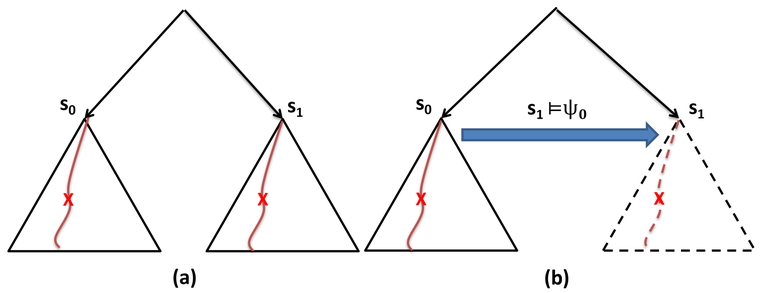
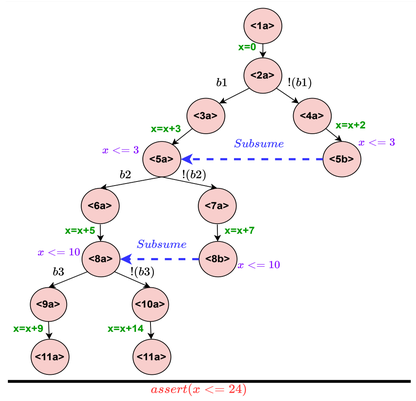


Figure 2. Exploration of Symbolic Execution Tree in Non-pruning DSE vs. Pruning DSE

Symbolic Execution Tree with Interpolation



Consider this program:

```
 x = 0;
if (b1) x += 3 else x += 2
if (b2) x += 5 else x += 7
if (b3) x += 9 else x += 14
assert(x <= 24)
```

- DFS traversal.
- Without interpolation: The full tree is traversed.
- With interpolation:
  1. $\langle 8b \rangle$ context contains $x = 10$. It is subsumed with the tree interpolant from $\langle 8a \rangle$: $x \leq 10$.
  2. $\langle 5b \rangle$ context contains $x = 2$. Subsumed with the tree interpolant from $\langle 5a \rangle$: $x \leq 3$.
  3. Big subtree traversal is avoided.

Interpolation: Weakest Precondition

1. PATH Interpolant: These interpolants are generated for each path during backward tracking.
2. TREE Interpolant : They are computed as the conjunction of PATH interpolants.

- Ideal interpolant is the weakest precondition (WP) of the target. Unfortunately, WP is intractable to compute, which means it is difficult or impossible to find an exact solution.
- For example, in the above code snippet: assume $(not(b1 \wedge \neg b2 \wedge \neg b3))$.
- Hence, the WP before the first if-statement is:
  WP is: $b1 \longrightarrow (\neg b2 \wedge b3 \wedge x \leq 7) \vee (b2 \wedge x \leq 4)$
  $\neg b1 \longrightarrow x < 3$
- Essentially, WP is exponentially disjunctive. This means that any one of the conditions can be satisfied for the target to be reached. (As shown here)
- One way to approximate WP is to use a conjunctive approximation, which involves expressing the WP as a conjunction of simpler conditions.

Approximation of Weakest Precondition

A Path is a sequence of assignments and assume instructions:

1. Interpolant of Assignment instruction:
   - wp$(inst, \omega) = \cdots$ inverse transition of $inst$ over ω
   - Implemented at LLVM IR level: LD/ST, add, sub, cmp, cast, GEP, etc.
   - e.g. $\omega : x \leq 15$ and $inst : x = z + 2$, then wp$(inst, \omega) : z \leq 13$
2. Interpolant of Assume instruction (C is incoming Context):
   $\{C\}$ assume(B) $\{\omega\}$
   - WP Approximation: find $\bar{C}$ to replace C
   - ABDUCTION PROBLEM !!!

This algorithm is the heart of TracerX:

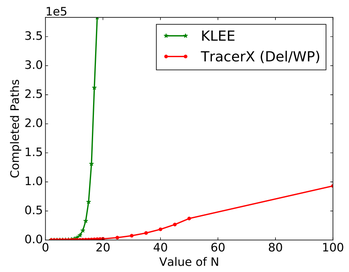
1. We compute finest partition so that $var(C_i) * var(C_j)$ $s.t.$ $i \neq j$: $\{C_1 * C_2 * C_3 * ... * C_n\}$ assume(B) $\{\omega_1 * \omega_2 * \omega_3 * ... * \omega_m\}$ ($*$ is as in separation logic).
2. Bunch C_i into three:

- Target independent: The C_i which are separate from B and ω.
  Action: Replace C_i with $true$, i.e. remove C_i.
- Guard independent: Consider $C_{gi} \equiv C_i$ s.t. $C_i * B$; and, $\omega_{gi} \equiv \omega_j$ s.t. $B * \omega_j$.
  Action: Replace C_{gi} by ω_{gi}.
- Remainder of the C_i: We do not capture exact WP for this group.
  e.g. {z == 5} assume$(x > z - 2)$ $\{x > 0\}$ (Here, $z > 2$ is the WP)
  Action: No change to C_i, i.e. keep C_i.

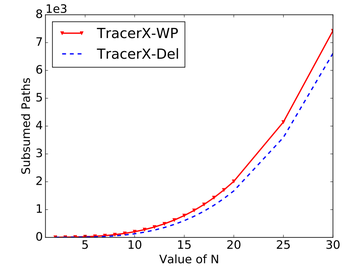
KLEE [1] v/s TracerX-Del [3] v/s TracerX-WP [4]

Consider this program; here, function `f(N)`, returns the sum of the first `N` natural numbers.

```
int counter=0; char input[N+1];
klee_make_symbolic(input,(N+1)*
sizeof(char),"input");
for (int i=0;i<=N; i++){
    if(input[i]==B)
        counter=counter+1;
    else
        counter+=i; }
klee_assert(counter!=f(N)+1);
```



- KLEE suffers from path explosion and hits the timeout for N=17.
- Both TracerX-Del and TracerX-WP scale up to N=50 [timeout=600 secs for each tool]
- Clear gap in the number of subsumed paths between TracerX-Del and TracerX-WP.
- TracerX-WP generates better interpolants than TracerX-Del for subsumption.
- Reason: TracerX-Del generates interpolants as a subset of the incoming context whereas TracerX-WP generates interpolants from the weakest precondition of a path.



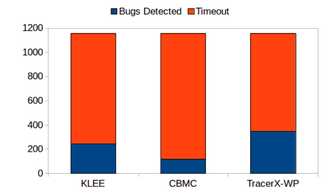
Experimental Results

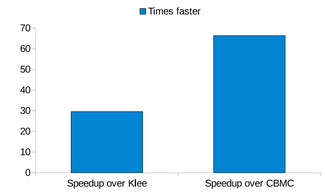
Data set: All C-programs from RERS-2012 Challenge [6].

- **Total targets:** 1159
- All three systems **KLEE** [1], **CBMC** [5] and **TracerX-WP** [4] are run for 3600 seconds

Observations:

1. TracerX-WP able to detect 348 targets, while KLEE and CBMC are detected 245 and 117 targets respectively.
2. All the targets reached by TracerX-WP are super set of targets covered by CBMC and KLEE.
3. TracerX-WP is 29.59x faster than KLEE and 66.37x faster than CBMC





References

[1] C. Cadar et al. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, 2008.

[2] J. Jaffar et al. TRACER: A symbolic execution tool for verification. In: CAV, 2012.

[3] J. Jaffar et al. TracerX: Dynamic symbolic execution with interpolation (competition contribution). In: FASE, 2020.

[4] A. Dutta et al. TracerX: Pruning Dynamic Symbolic Execution with Deletion and Weakest Precondition Interpolation (competition contribution). In: FASE, 2024.

[5] D. Kroening D et al. CBMC-C Bounded Model Checker. In: TACAS 2014.

[6] http://rers-challenge.org/